Algorithm and Model Changes

magnetic-field-inspired-navigation

2026-05-22

This document records major algorithm and model decisions in the cleanroom `magnetic-field-inspired-navigat` implementation.

Rule for this repo

- Any major change to the controller, dynamics model, obstacle model, sensing abstraction, or mathematical formulation must be recorded in this file in the same change that introduces it.

The purpose is to keep a durable map between:

- the legacy ROS implementation in `double_integrator`
- the cleanroom Python implementation
- intentional simplifications or deviations

Entry 2026-05-22: Initial Cleanroom Double-Integrator Reference

Status

- Implemented in `src/mfinav/double_integrator.py`
- Scenario runner in `scripts/run_reference.py`

Legacy source anchors

- Goal controller and relaxation: `nodes/ataka_listener.py`
- Duplicate goal controller: `nodes/multi_agent_listener.py`
- Magnetic avoidance, `field == 2`: `nodes/collision_avoidance_okt.py`
- Double-integrator rollout: `nodes/system_motion.py`

What Was Implemented

The current cleanroom reference includes:

- 2D double-integrator dynamics
- 2D circular obstacle abstraction
- goal-relaxation gain logic inspired by the ROS implementation
- magnetic-field-inspired obstacle avoidance based on the ROS `field=2` branch

- additive total command

$$a = a_{\text{goal}} + a_{\text{obs}} \tag{1}$$
$$v_{k+1} = v_k + a_k \Delta t \tag{2}$$
$$p_{k+1} = p_k + v_{k+1} \Delta t \tag{3}$$

Goal Relaxation Implemented

Let

- $p \in \mathbb{R}^2$ be robot position
- $v \in \mathbb{R}^2$ be robot velocity
- $g \in \mathbb{R}^2$ be goal position
- $r_g = g - p$ be the goal vector
- r_o be the closest obstacle vector
- $d_g = \|r_g\|$
- $d_o = \|r_o\|$

The relaxation scale used in the cleanroom implementation is

$$k_{\text{relax}} = \left(1 - e^{-d_o/1.5}\right) \cdot w_{\angle} \cdot w_{\text{prog}} \cdot w_{\text{exit}} \tag{4}$$

where

$$w_{\angle} = 1 - \cos\theta = 1 - \frac{r_g^\top r_o}{\|r_g\|\|r_o\|} \tag{5}$$

$$w_{\text{prog}} = \begin{cases} 1, & d_g \le d_{g,0} \\ \exp\left(-\dfrac{d_g - d_{g,0}}{0.1}\right), & d_g > d_{g,0} \end{cases} \tag{6}$$

and

$$w_{\text{exit}} = \exp\left(-\frac{d_g - d_{g,\min}}{0.1}\right) \tag{7}$$

with:

- $d_{g,0}$: initial distance to the goal
- $d_{g,\min}$: smallest distance to the goal reached so far

This follows the structure of the ROS code around the `goal_relax == 1` block.

Goal Controller Implemented

For the far field, the current cleanroom implementation uses a simplified speed-and-turn controller inspired by the ROS geometric block:

$$a_{\text{goal}} = a_{\text{attr}} + a_{\text{turn}} \tag{8}$$

$$a_{\text{attr}} = -\left(k_p\, k_{\text{relax}}\right)\left(\|v\| - v_{\max}\right)\hat{s} \tag{9}$$

where $\hat{s}$ is:

- the normalized velocity direction if $\|v\| > 0$
- otherwise the normalized goal direction

The turning term is

$$a_{\text{turn}} = k_\omega\, k_{\text{relax}}\, \|v\|\, \hat{n}\, (\hat{n}^\top \hat{g}) \tag{10}$$

where:

- $\hat{g} = r_g/\|r_g\|$
- $\hat{n}$ is the left-perpendicular of the current heading

Near the goal, the controller switches to a PD-like attractive form:

$$a_{\text{goal}} = k_{p,\text{goal}} r_g - k_d v \tag{11}$$

Magnetic-Field-Inspired Obstacle Term Implemented

The cleanroom port uses the `field == 2` structure from the old code.

Let

- r_o be the closest vector from robot to obstacle surface
- $d = \|r_o\|$
- $\hat{l}_a = v/\|v\|$ be the normalized robot motion direction
- $r_{\text{from obs}} = -r_o$
- $\hat{r} = r_{\text{from obs}}/\|r_{\text{from obs}}\|$

The obstacle current direction is the tangential projection of the robot motion:

$$\hat{t} = \hat{l}_a - (\hat{l}_a^\top \hat{r})\hat{r} \tag{12}$$

If this becomes degenerate, the implementation falls back to a perpendicular direction.

The 2D magnetic field surrogate is modeled as

$$B \propto \frac{\|v\|}{d}\,\hat{t}_{\perp} \tag{13}$$

and the magnetic avoidance term is

$$F_{\text{mag}} = k_m \hat{l}_{a,\perp}(\hat{l}_{a,\perp}^{\top} B) \tag{14}$$

This is a 2D analogue of the nested cross-product structure in the ROS code:

$$B = [\hat{t}]_{\times} v/d \tag{15}$$

$$F = k_m [\hat{l}_a]_{\times} B \tag{16}$$

Added Repulsive Correction Implemented

The cleanroom implementation also keeps the old additional repulsive term near the obstacle:

$$F_{\text{rep}} = -k_r \left(\frac{1}{d} - \frac{1}{d_b}\right)\frac{r_o}{d} \quad \text{for } d \le d_b \tag{17}$$

Then it is projected to remove the component along the motion direction:

$$F_{\text{rep}}^{\perp} = F_{\text{rep}} - (F_{\text{rep}}^{\top}\hat{l}_a)\hat{l}_a \tag{18}$$

The final obstacle term is:

$$a_{\text{obs}} = F_{\text{mag}} + F_{\text{rep}}^{\perp} \tag{19}$$

Major Simplifications Relative To The Old ROS Code

These are intentional and important:

1. The current reference is strictly 2D. The ROS code contains both 2D and 3D branches.
2. Obstacles are currently circles. The ROS stack computes obstacle geometry from point clouds, polygonal segments, and local obstacle surfaces.
3. The far-field goal controller is structurally inspired by the ROS code, but it is not yet a line-by-line reproduction of the SO(3)-style geometric block.
4. The magnetic term is mathematically faithful in structure, but the current implementation uses a compact 2D analogue rather than a direct symbolic port of the original 3D cross-product expressions.
5. The cleanroom model removes all ROS-specific concerns: TF, publishers/subscribers, launch files, point-cloud nodes, timestamps, and platform adapters.

Why This Change Was Made

The goal of this first cleanroom step is verification by isolation:

- keep the algorithm small enough to reason about
- remove ROS coupling
- make rollout behavior inspectable with CSV and plot outputs
- establish a stable place for future refactors and comparisons

Required Follow-Up

Future major changes should update this file, especially if they modify:

- the exact goal-relaxation gain
- the magnetic obstacle formula
- the near-goal switching logic
- the state update model
- the obstacle abstraction
- the benchmark scenario assumptions

Entry 2026-05-22: Structural Refactor Toward The 2022 Paper

Status

- Refactored in `src/mfinav/double_integrator.py`
- This is a structural alignment step, not yet a full equation-level reconstruction of the 2022 paper

Motivation

The 2022 paper describes obstacle avoidance as the combination of two terms:

$$F_o = F_b + F_a \tag{20}$$

where:

- F_b is the boundary-following field
- F_a is the collision-avoidance field

The earlier cleanroom prototype merged these ideas into one obstacle term plus an added repulsive correction. That made the code easy to test, but it hid the paper's core decomposition.

What Changed

The obstacle logic is now split into explicit components:

- `LocalSensingModel`
- `BoundaryFollowingField`

- `CollisionAvoidanceField`

The total control now has the explicit structure

$$a = a_{\text{goal}} + a_b + a_a \tag{21}$$

where:

- a_b is the boundary-following component
- a_a is the collision-avoidance component

Observation Model

The refactor introduces a local observation container:

$$\mathcal{O} = \{r_o,\ d_o\} \tag{22}$$

where:

- r_o is the closest-obstacle vector
- $d_o = \|r_o\|$

This is still generated from an analytic circle model in the current code, but the new interface is intentionally shaped like a local sensing output so it can be replaced later by sensor-driven obstacle processing.

Boundary-Following Field

The boundary-following component keeps the same current prototype behavior, but it is now isolated as its own field:

$$\hat{t} = \hat{l}_a - (\hat{l}_a^\top \hat{r})\hat{r} \tag{23}$$

$$B \propto \frac{\|v\|}{d}\hat{t}_\perp \tag{24}$$

$$a_b = k_m \hat{l}_{a,\perp}(\hat{l}_{a,\perp}^\top B) \tag{25}$$

This is still the compact 2D analogue used previously, but it is now clearly identified as the boundary-following term.

Collision-Avoidance Field

The repulsive near-obstacle correction is now made explicit as the collision-avoidance field:

$$a_a = -k_r \left(\frac{1}{d} - \frac{1}{d_b}\right)\frac{r_o}{d} \quad \text{for } d \le d_b \tag{26}$$

and, when motion direction is defined, it is projected to remove the component parallel to the current motion:

$$a_a^{\perp} = a_a - (a_a^{\top}\hat{l}_a)\hat{l}_a \tag{27}$$

Goal Relaxation Status

The goal-relaxation logic remains available, but it is now explicitly marked as a legacy extension from the ROS code rather than a confirmed core part of the 2022 paper implementation.

This is represented by a configuration switch:

$$\texttt{use\_legacy\_goal\_relaxation} \in \{0, 1\} \tag{28}$$

When disabled, the goal controller uses unit scaling rather than the legacy relaxation factor.

What This Refactor Improves

- The code structure now mirrors the paper's decomposition more clearly.
- It becomes easier to replace the current placeholder formulas with paper-faithful ones.
- It isolates local sensing, boundary following, and collision avoidance as separate concerns.

What Still Does Not Match The 2022 Paper Exactly

- The exact paper equations for F_b and F_a are not yet reconstructed line by line.
- The threshold semantics of r_l and r_{la} are not yet implemented as separate paper-faithful activations.
- The Section 4.4 closest-point averaging extension is still missing.
- The current observation model still comes from an analytic obstacle, not real local sensing.

Entry 2026-05-22: Benchmark Environments and APF Baseline

Status

- Added multi-obstacle environment support
- Added convex and non-convex benchmark scenarios
- Added a standard artificial potential field (APF) baseline
- Added a benchmark runner that saves trajectory plots and metrics

What Changed

The simulator is no longer restricted to a single obstacle. A new obstacle collection abstraction returns the closest obstacle vector among several primitive obstacles:

$$r_o = \arg\min_{r_i \in \mathcal{R}} \|r_i\| \tag{29}$$

where $\mathcal{R}$ is the set of closest-point vectors returned by all obstacles in the environment.

This enables more realistic environments while preserving the local-sensing structure used by the controller.

To make controller comparisons numerically meaningful, the simulator now clips both commanded acceleration and robot speed:

$$a \leftarrow \begin{cases} a, & \|a\| \le a_{\max} \\ a_{\max}\dfrac{a}{\|a\|}, & \text{otherwise} \end{cases} \tag{30}$$

$$v \leftarrow \begin{cases} v, & \|v\| \le v_{\max} \\ v_{\max}\dfrac{v}{\|v\|}, & \text{otherwise} \end{cases} \tag{31}$$

The rollout also terminates on collision, approximated by zero obstacle clearance in the simplified geometric model.

New Scenario Types

Three benchmark scenarios are now defined:

- a single convex obstacle
- a cluster of convex obstacles
- a non-convex U-shaped obstacle approximated by several circles

The non-convex environment is still a geometric approximation, but it is useful for testing whether the controller exhibits boundary-following behavior beyond simple single-obstacle deflection.

APF Baseline

A standard artificial potential field baseline is added with the structure

$$a_{\text{APF}} = a_{\text{att}} + a_{\text{rep}} \tag{32}$$

where the attractive term is

$$a_{\text{att}} = K_P(p_g - p) - K_D v \tag{33}$$

and the repulsive term is activated near obstacles:

$$a_{\text{rep}} = -K_R\left(\frac{1}{d} - \frac{1}{d_b}\right)\frac{r_o}{d^2} \quad \text{for } d \le d_b \tag{34}$$

This baseline is not meant to reproduce every APF variant in the literature; it is meant to provide a simple and interpretable comparison against the current MFI implementation.

Verification Outputs

The benchmark runner now produces:

- trajectory comparison plots

- CSV metrics for each scenario and method

The current summary metrics are:

- success flag
- collision flag
- number of steps
- path length
- final goal distance
- minimum obstacle clearance
- mean speed

Why This Matters

This change gives us a repeatable way to evaluate:

- whether the current MFI implementation handles more complex geometry
- whether it offers any practical benefit over a standard APF baseline
- where the current simplified implementation breaks down

Entry 2026-05-22: Paper-Driven Thresholds, Averaging Extension, and Legacy Goal Relaxation Demotion

Status

- Added explicit threshold semantics for boundary following and collision avoidance
- Added the Section 4.4 style closest-point averaging extension
- Changed legacy goal relaxation from default-on to optional

Thresholded Obstacle Activation

The implementation now follows the paper structure more closely by separating the activation thresholds:

- boundary-following field active for $d \leq r_l$
- collision-avoidance field active for $d \leq r_{la}$

with default values:

$$r_l = 4.0, \qquad r_{la} = 2.5 \tag{35}$$

This replaces the previous use of generic proximity constants that did not map cleanly to the paper's notation.

The implementation now also applies smooth activation weights:

$$\sigma_b = \text{clip}\left(\frac{r_l - d}{r_l - r_{la}}, 0, 1\right) \tag{36}$$

$$\sigma_a = \text{clip}\left(\frac{r_{la} - d}{r_{la}}, 0, 1\right) \tag{37}$$

so that the effective fields are:

$$a_b \leftarrow \sigma_b a_b \tag{38}$$

$$a_a \leftarrow \sigma_a a_a \tag{39}$$

This makes the obstacle interaction less abrupt and improved the clustered obstacle benchmark in practice.

Closest-Point Averaging Extension

To approximate the Section 4.4 extension for non-unique closest points, the observation model now collects all sensed obstacle vectors whose magnitudes are within a threshold δ_r of the minimum sensed distance:

$$\mathcal{R}_\delta = \{r_i \mid \|r_i\| \leq r_{\min} + \delta_r\} \tag{40}$$

The averaged vector is then

$$\bar{r} = \frac{1}{|\mathcal{R}_\delta|} \sum_{r_i \in \mathcal{R}_\delta} r_i \tag{41}$$

Following the paper's heuristic:

- if $\|\bar{r}\| < \|r_o\|$, use $\bar{r}$ as the working closest-point vector
- otherwise, use the standard closest vector r_o

This gives the code a practical non-unique closest-point handling mechanism for concave and sharp-corner approximations.

To better reflect the paper's sensing assumption, the implementation no longer uses only one exact closest-point vector per obstacle. Each obstacle now provides multiple local surface samples around the nearest visible region, and the averaging extension operates over these sensed vectors rather than over a single analytic point per obstacle.

Small-Current Fallback

The paper notes that the projected artificial current can become very small when the robot direction is nearly aligned with the robot-to-obstacle vector. To mitigate this, the implementation now checks the magnitude of the projected current and falls back to a unit tangent surrogate when:

$$\|\hat{t}\| < \epsilon \tag{42}$$

with default

$$\epsilon = 3 \times 10^{-6} \tag{43}$$

Legacy Goal Relaxation

The ROS-derived goal-relaxation term is no longer the default path for the implementation. It is now explicitly treated as an optional legacy extension:

$$\texttt{use\_legacy\_goal\_relaxation} = 0 \tag{44}$$

by default.

When enabled, the implementation restores the earlier ROS-inspired scaling factor. When disabled, the goal controller operates without that legacy gain modulation.

Why This Change Was Made

These changes address three of the most important mismatches between the prototype and the 2022 paper:

- explicit r_l and r_{la} threshold behavior
- handling of non-unique closest points
- clearer separation between paper-driven behavior and ROS-legacy behavior

Entry 2026-05-22: Tuned Defaults and Stronger Evaluation Metrics

Status

- Increased the default rollout horizon from 2000 to 6000 steps
- Tuned the default interaction and sensing parameters around the better-performing region
- Added time-to-goal and path-efficiency evaluation metrics

Tuned Defaults

The default parameters are now:

$$r_l = 4.0, \quad r_{la} = 2.5, \quad c_{\text{field}} = 12.0, \quad c_{\perp} = 35.0 \tag{45}$$

and the local sensing surrogate uses:

$$N_{\text{samples}} = 15, \qquad \theta_{\text{window}} = 1.0, \qquad \delta_r = 0.5 \tag{46}$$

These values were selected because they materially improved the convex-cluster benchmark while preserving good behavior in the single-convex case.

With the longer rollout horizon, the current MFI implementation is able to reach the success region in all three default benchmark scenarios.

Additional Evaluation Metrics

The benchmark harness now reports:

- time-to-goal in simulation steps
- path efficiency

with

$$\text{path efficiency} = \frac{\|p_f - p_0\|}{L} \tag{47}$$

where:

- p_0 is the initial position
- p_f is the realized final position
- L is the realized trajectory length

This definition ensures the metric remains meaningful even when a method does not reach the goal.

Time-to-goal is defined as the first step index where:

$$\|p - p_g\| \leq r_{\text{success}} \tag{48}$$

If the goal region is never reached, the value is reported as ∞.

Entry 2026-05-22: Polygonal Obstacles With Local Sensing

Status

- Added explicit polygon obstacle support
- Kept the same local-sensing and closest-point averaging pipeline
- Converted the default benchmark environments to polygonal convex and non-convex obstacles

Polygon Obstacle Model

A polygon obstacle is represented by an ordered list of boundary vertices

$$\mathcal{P} = \{v_1, v_2, \ldots, v_N\} \tag{49}$$

with edges formed by consecutive vertex pairs. For a robot position p, the closest-point vector is computed by projecting onto each boundary segment and selecting the shortest resulting vector.

$$r_o = \arg\min_{r_i \in \mathcal{E}} \|r_i\| \tag{50}$$

where $\mathcal{E}$ is the set of candidate vectors to all polygon edges.

Local Sensing on Polygon Boundaries

Instead of using only a single exact closest point, the polygon model generates multiple local samples along each edge around the nearest visible region. These sampled boundary vectors are passed into the same observation model already used for circular obstacles:

- find the minimum sensed distance
- gather all vectors within the averaging band δ_r
- compute the averaged closest-point surrogate
- use the averaged vector when it is shorter than the single closest vector

This keeps the paper-style local sensing and closest-point averaging intact while making convex and non-convex geometry easier to define.

Why This Matters

Polygonal obstacles let us represent:

- single convex obstacles such as boxes or rotated quadrilaterals
- clustered convex environments
- explicitly non-convex obstacles such as U-shaped boundaries

without approximating every case by a large collection of circles.

Entry 2026-05-22: Signed Clearance Evaluation and Head-On Boundary Fix

Status

- Replaced unsigned polygon distance checks with signed obstacle clearance
- Added explicit safety-violation reporting in addition to hard collision reporting
- Changed the polygon boundary-following and repulsion terms so a head-on wall encounter no longer degenerates to zero avoidance command

Signed Clearance

For evaluation we now distinguish between:

- *distance to the sensed boundary* used by the field law
- *signed clearance* used by the evaluator

For circles,

$$c(p) = \|p - p_c\| - r \tag{51}$$

For polygons, the evaluator computes the minimum Euclidean distance from the robot position p to any boundary edge and assigns a negative sign if p lies inside the polygon:

$$c(p) = \begin{cases} -\min\limits_{e \in \partial\mathcal{P}} d(p, e), & p \in \mathcal{P} \\ \min\limits_{e \in \partial\mathcal{P}} d(p, e), & p \notin \mathcal{P} \end{cases} \tag{52}$$

This means:

- $c(p) \leq 0$ indicates collision / penetration
- $0 < c(p) \leq c_{\text{safety}}$ indicates an unsafe low-clearance path

Updated Success Accounting

The benchmark now reports two success-style quantities:

$$\text{goal\_reached\_once} = \begin{cases} 1, & \exists k \text{ such that } \|p_k - p_g\| \leq r_{\text{success}} \text{ and } c_{\min} > c_{\text{safety}} \\ 0, & \text{otherwise} \end{cases} \tag{53}$$

$$\text{success} = \begin{cases} 1, & \|p_f - p_g\| \leq r_{\text{success}} \text{ and } c_{\min} > c_{\text{safety}} \\ 0, & \text{otherwise} \end{cases} \tag{54}$$

The first quantity captures whether the trajectory ever reached the goal region, while the second remains the stricter final-state metric.

Why The Old Polygon Case Failed

The earlier 2D boundary-following and repulsion approximation had a degeneracy for a head-on encounter with a vertical wall:

- the tangential term collapsed to zero because the projected field and the projected motion-perpendicular direction became orthogonal
- the repulsive term collapsed to zero because the full normal repulsion was projected away along the current motion direction

That is why parameter sweeps alone could not fix the U-shaped polygon case.

Updated Field Form

To remove that degeneracy, the boundary-following term now directly uses the local tangent estimate:

$$a_b = \sigma_b \, c_{\text{field}} \, v_{\text{guide}} \, \frac{\hat{t}}{d} \tag{55}$$

where

$$v_{\text{guide}} = \max(\|v\|, 0.5 \, v_{\max}) \tag{56}$$

and $\hat{t}$ is the normalized local tangent estimate (or a perpendicular fallback when the projected tangent becomes degenerate).

The collision-avoidance term now keeps its full normal component:

$$a_a = -\sigma_a \, c_\perp \left(\frac{1}{d} - \frac{1}{r_{la}} \right) \frac{r_o}{d} \tag{57}$$

This change is less "projected" than the previous 2D approximation, but it is much more appropriate for polygonal wall encounters because it preserves a real push-away component.

Entry 2026-06-18: Closer Reconstruction of Paper PD, Geometric, F_b, and F_a

Status

- Added separate paper-style PD and paper-style geometric presets
- Replaced the earlier paper placeholder F_b with a closer cross-product-based reconstruction
- Replaced the earlier smoothed paper placeholder F_a with a hard-threshold projected repulsive term
- Added memory of the previously proposed obstacle-surface current for the small-current degeneracy case

Paper-style Goal Modes

The repository now distinguishes:

- `paper_pd`: direct attractive PD goal control
- `paper_geometric`: speed regulation plus steering correction
- `pragmatic`: the cleaner benchmark-oriented hybrid controller used in earlier refactors

For `paper_pd`, the implemented goal term is:

$$u_g = K_P(p_g - p) - K_D \dot{p} \tag{58}$$

For `paper_geometric`, the far-field speed regulation remains:

$$u_{\text{attr}} = K_{P,r}(v_{\max} - \|v\|)\hat{v} \tag{59}$$

and the steering term is now treated as a 2D analogue of the old SO(3)-based rotation controller:

$$u_{\text{steer}} = \omega \, v_\perp, \qquad \omega = K_\omega \, \text{angle}(v, p_g - p) \tag{60}$$

Paper-style Boundary-Following Field

The previous "paper mode" used a smoothed 2D surrogate. The updated implementation is closer to the old ROS magnetic construction and to the paper derivation:

$$l_o = l_a - \left(l_a^\top \hat{r} \right) \hat{r} \tag{61}$$

$$B = [l_o]_\times v/d \tag{62}$$

$$F_b = c[l_a]_\times B \tag{63}$$

The implementation now uses a hard activation at $d \leq r_l$ for paper mode instead of the earlier smooth activation weight.

Small-current degeneracy handling

When $\|l_o\|$ becomes too small, the implementation now reuses the previously proposed normalized obstacle-surface current instead of immediately switching to the pragmatic tangent fallback. This follows the paper discussion more closely than the earlier heuristic.

Paper-style Collision-Avoidance Field

For paper mode, the implementation now applies:

$$F_a = -c_\perp \left(\frac{1}{d} - \frac{1}{r_{la}}\right) \frac{r_o}{d} \tag{64}$$

with a projection that removes the component along the instantaneous robot motion direction in order to better preserve the paper property that F_a does not change speed.

Observed outcome

These changes made the paper-aligned PD controller much more plausible on the simple and clustered convex polygon scenarios. The paper-aligned geometric mode also became stable after correcting the steering sign in the 2D analogue.

However, the paper-aligned modes still collide in the non-convex U-shaped polygon benchmark under the current local polygon sensing abstraction, so the equation-level reconstruction is improved but still not complete.

Entry 2026-06-18: Matching Paper Actuation, Goal Relaxation, and Local Sensing Assumptions

Status

- Paper-aligned modes now run without acceleration or speed clipping
- Goal relaxation is enabled again for the paper-aligned modes
- Local sensing for paper-aligned modes now uses ray-based sensing instead of direct analytic boundary sampling

Actuation Assumption

The 2022 paper assumes non-saturatable actuators. Earlier versions of the clean repository applied norm clipping to both acceleration and speed even in the paper-style modes. This has now been removed for the paper-aligned presets by setting:

$$\|u\|_{\max} = \infty, \qquad \|v\|_{\max} = \infty \tag{65}$$

for `paper_pd` and `paper_geometric`.

Goal Relaxation Assumption

Goal relaxation is now active again in the paper-aligned presets. The current implementation still uses the inherited ROS-era weighting logic, but the paper modes no longer bypass goal relaxation entirely.

Local Sensing Assumption

Earlier versions used direct local boundary sampling from known analytic circle or polygon geometry. The paper-aligned sensing path now uses a synthetic range-sensor style model based on ray casting:

- rays are cast around the robot over a full angular sweep
- for each ray, the nearest obstacle intersection within the sensor range is selected
- only those currently visible local obstacle points are passed to the closest-point and averaging logic

This is still a simulation abstraction, but it matches the paper's "temporally and spatially local sensing" assumption much better than the earlier direct-boundary access.

Observed outcome

These assumption changes improved the paper-aligned controllers substantially:

- the paper-aligned PD controller now succeeds in the clustered convex polygon benchmark
- it now succeeds in the clustered convex polygon benchmark
- the paper-aligned geometric controller also succeeds in the clustered convex polygon benchmark
- both paper-aligned modes remain collision-free in the non-convex U-shaped benchmark
- the paper-aligned geometric controller approaches the non-convex U-shaped benchmark goal much more closely while maintaining safe clearance

However, the paper-aligned PD controller still fails to make substantial progress in the non-convex U-shaped benchmark and the paper-aligned geometric controller still stops short of full convergence. This suggests that the remaining discrepancy is no longer just about actuation, goal relaxation, or local sensing; it is now more likely tied to the exact controller equations and the polygon benchmark abstraction itself.

Entry 2026-06-18: Explicit 2D Derivation Target

Status

- Added a dedicated 2D derivation target document for the paper equations
- Fixed a concrete target form for l_a, n_o, l_o, $l_o^{\perp}$, F_b, and F_a
- Established the exact 2D target to implement against in the next phase

Why This Was Needed

The 2022 paper is written in a general 3D vector-field form, while the current cleanroom benchmark is planar. That left too much room for interpretation in the 2D implementation. The new derivation note reduces the paper equations to a precise 2D target before further controller refactors are attempted.

Document Added

- `docs/paper_2d_derivation_target.tex`

2D Target Highlights

The derivation note fixes the following working targets:

$$n_o = -\frac{r_o}{\|r_o\|} \tag{66}$$

$$\tilde{l}_o = (I - n_o n_o^\top) l_a \tag{67}$$

$$l_o = \begin{cases} \dfrac{\tilde{l}_o}{\|\tilde{l}_o\|}, & \|\tilde{l}_o\| \geq \epsilon \\ l_{o,\mathrm{prev}}, & \|\tilde{l}_o\| < \epsilon \end{cases} \tag{68}$$

$$l_o^\perp = -l_o \tag{69}$$

Using the planar rotation matrix

$$J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \tag{70}$$

the working 2D target for the boundary-following field is

$$F_b^{2D} = -c \frac{l_o \times_2 v}{\|r_o\|} J l_a \tag{71}$$

and the working 2D target for the collision-avoidance field is

$$F_a^{2D} = -c_\perp \frac{l_o^\perp \times_2 r_o}{\|r_o\|} J l_a \tag{72}$$

Caution

The derivation note is now the implementation target, not yet the final proof that the current code exactly matches the paper. In particular, the F_a expression still needs to be checked carefully against the paper equations and the ROS implementation.

Entry 2026-06-18: Step 2 – F_b Matched to the Explicit 2D Target

Status

- Replaced the paper-mode boundary-following field implementation with the direct 2D target formula
- Removed the embedded-3D surrogate path for the paper-mode F_b
- Verified that benchmark behavior remained stable after the change

New paper-mode F_b implementation target

The paper-mode boundary-following field is now implemented directly as:

$$F_b^{2D} = -c\frac{l_o \times_2 v}{\|r_o\|} J l_a \tag{73}$$

where:

$$J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{74}$$

and l_o is the obstacle-surface current obtained from the 2D projection target defined in `paper_2d_derivation_targ`

Why This Matters

The earlier implementation used an embedded 3D cross-product calculation that was consistent in spirit, but still left ambiguity about whether the code matched the exact intended 2D reduction. The new implementation makes the 2D magnetic field law explicit and removes that ambiguity for F_b.

Observed outcome

Benchmark behavior remained effectively unchanged after replacing the old paper-mode F_b with the direct 2D target formula. This is a good sign because it suggests the derivation target and the previous cross-product-based implementation were already closely aligned for F_b.

Entry 2026-06-18: Step 3 – F_a Matched to the Current 2D Target and Failed

Status

- Replaced the paper-mode collision-avoidance field with the direct 2D target form based on $l_o^{\perp}$
- Reused the same paper-style surface-current construction and ϵ-memory rule as F_b
- Observed a strong degradation in benchmark behaviour

Applied target

The paper-mode collision-avoidance field was updated to the current 2D target:

$$F_a^{2D} = -c_\perp \frac{l_o^\perp \times_2 r_o}{\|r_o\|} J l_a \tag{75}$$

Observed outcome

Unlike Step 2, this replacement did *not* preserve behaviour:

- paper-aligned geometric behaviour became unstable in the convex benchmark
- paper-aligned PD lost success in the clustered convex benchmark under the current safety criterion
- both paper-aligned modes collided again in the non-convex U-shaped benchmark

Interpretation

This is important negative evidence. It strongly suggests that the current 2D target for F_a, while structurally paper-shaped, is still not the correct 2D reduction of the paper's intended field law. In other words:

- matching F_a to the *current* target was useful,
- but the *target itself* is probably incomplete or incorrect,
- especially regarding the exact scalar function and the role of $l_o^\perp$ in the paper construction.

Consequence

The next step should not be parameter tuning. The next step should be to revise the 2D derivation target for F_a itself by checking the paper equations and the ROS implementation more carefully.

Entry 2026-06-18: Step 4 – F_a Revised Back to a ROS-Backed Short-Range Projected Repulsion

Status

- Rejected the previously tried magnetic $l_o^\perp$-based 2D target for F_a
- Restored the paper-mode collision term to a short-range radial repulsion
- Kept the orthogonal projection onto the subspace perpendicular to the motion direction
- Updated the 2D derivation note so it no longer claims the failed magnetic target is the working F_a reduction

Applied target

The paper-mode collision-avoidance field is now:

$$\tilde{F}_a = -c_\perp \left(\frac{1}{\|r_o\|} - \frac{1}{r_{la}} \right) \frac{r_o}{\|r_o\|} \tag{76}$$

$$F_a^{2D} = \left(I - l_a l_a^\top \right) \tilde{F}_a \tag{77}$$

Why this change was necessary

The ROS `field=2` branch does not support the previously assumed $l_o^{\perp}$-magnetic reduction for F_a. The stronger implementation evidence is:

- the magnetic cross-product structure already belongs to the boundary-following term,
- the close-range collision term is built from the obstacle vector itself,
- the result is then projected so the final command remains orthogonal to the motion direction.

Interpretation

This makes the clean Python implementation more faithful to the old ROS behavior and removes a target that was contradicted by both benchmarks and code audit. It does *not* yet prove that the paper's printed F_a equation has been reconstructed exactly, but it is a materially better and better-tested target than the failed magnetic alternative.

Observed outcome

After this revision:

- both paper-aligned modes recovered clean success in the convex polygon benchmarks,
- the non-convex U-shaped case returned to collision-free behavior,
- the remaining gap is now goal convergence in the non-convex benchmark, not immediate collision.

Entry 2026-06-18: Added Non-Convex Diagnostic Instrumentation

Status

- Added per-step diagnostic capture for the paper-aligned controller components
- Added a dedicated U-shaped obstacle diagnostic script
- Re-ran the non-convex benchmark on the restored baseline to keep artifacts aligned with the current code

What was added

The clean Python implementation now records:

- the last local sensing observation,
- the goal-attraction command,
- the boundary-following field command,
- the collision-avoidance field command,
- the total command and goal-relaxation weight.

The new script `scripts/run_nonconvex_diagnostic.py` exports:

- a trajectory plot for the U-shaped benchmark,
- a per-step CSV with obstacle distance, goal distance, goal weight, heading error, and command norms.

Observed diagnostic findings

On the restored baseline:

- the paper PD controller stalls far before the goal with very small speed while still inside the sensing / avoidance band,
- the paper geometric controller gets close to the goal but keeps a small residual steering conflict and does not settle inside the success radius,
- a simple F_a taper experiment was tested and rejected because it regressed the convex-cluster benchmark.

Entry 2026-06-18: Goal Relaxation Switched to `used_obstacle_vector`

Status

- Updated the paper-style goal-relaxation path to use `used_obstacle_vector` instead of `closest_obstacle_vect`
- Re-ran the non-convex diagnostic and the full polygon benchmark suite

Motivation

The diagnostic traces suggested that the geometric controller's goal command remained small even after the obstacle terms had already become weak. One possible explanation was that goal relaxation was still using the wrong obstacle representative, since the paper-oriented sensing path already uses the averaged / selected vector `used_obstacle_vector` elsewhere.

Observed outcome

This change was largely neutral:

- the convex paper-mode benchmarks remained successful,
- the non-convex U-shaped benchmark changed only negligibly,
- therefore the choice between `closest_obstacle_vector` and `used_obstacle_vector` is not the main cause of the remaining non-convex failure.

Interpretation

This was still the correct paper-consistent cleanup to make. However, the remaining issue is more likely due to the structure of the paper-style goal relaxation itself, or to the interaction between that relaxation and the geometric far-goal controller, rather than to the obstacle representative alone.

Entry 2026-06-18: Geometric Controller Updated for Low-Speed Turning and Near-Goal Velocity Tracking

Status

- Strengthened the paper-style geometric far-goal steering term so it does not collapse when the robot speed becomes very small

- Replaced the geometric near-goal positional PD fallback with a distance-scaled desired-velocity tracking law
- Re-ran the U-shaped non-convex diagnostic and the full polygon benchmark suite

Motivation

Targeted diagnostics showed that:

- the geometric controller could get near the goal while obstacle terms were already weak,
- but the goal command itself became too small,
- and disabling goal relaxation did not resolve that failure.

This isolated the remaining issue to the structure of the geometric goal controller itself, especially in low-speed and near-goal conditions.

Applied change

Two updates were made to the paper-style geometric controller:

- the far-goal turning term now uses the motion direction basis instead of the raw velocity vector, so steering authority remains meaningful even when the speed is low,
- the near-goal branch now tracks a desired velocity whose magnitude is proportional to the remaining goal distance, instead of falling back to the weak positional PD term.

Observed outcome

This produced the first full non-convex success for the paper-style geometric controller:

- `paper_geometric` now succeeds in the U-shaped non-convex benchmark,
- the convex polygon benchmarks remain successful,
- `paper_pd` still remains the unsolved variant in the non-convex case.

Interpretation

This is strong evidence that the remaining geometric-mode issue was not mainly in F_a, F_b, or goal relaxation, but in the low-authority structure of the geometric goal controller near the end of the maneuver.

Entry 2026-06-18: Paper-PD Goal Gains Retuned for Non-Convex Convergence

Status

- Investigated the remaining paper-PD failure in the U-shaped non-convex benchmark
- Verified that disabling goal relaxation did not resolve the failure
- Performed a targeted paper-PD gain sweep
- Updated the default paper-PD gains to the best-performing safe candidate

Observed diagnosis

The targeted tests showed:

- the paper-PD controller still stalled in the non-convex case even when goal relaxation was disabled,
- therefore goal relaxation was not the main cause,
- some PD gain pairs did succeed, which indicated that the remaining issue was controller tuning rather than a fundamental structural mismatch.

Applied change

The default paper-PD gains were updated from $k_p = 0.04$, $k_d = 0.5$ to $k_p = 0.08$, $k_d = 0.5$.

The relaxed-speed gain parameter was updated consistently so the paper-style configuration remains internally coherent.

Observed outcome

With the updated default paper-PD gains:

- `paper_pd` succeeds in the single-convex polygon benchmark,
- `paper_pd` succeeds in the convex-cluster benchmark,
- `paper_pd` now also succeeds in the U-shaped non-convex benchmark,
- both paper-style controller variants now succeed across the current polygon benchmark set.

Interpretation

At the current 2D benchmark level, the remaining paper-PD issue was primarily a gain-tuning problem, not a goal-relaxation problem. This closes the previously open non-convex failure for the paper-style PD branch.

Entry 2026-06-19: Modular Refactor of Models, Navigators, Obstacles, Sensing, and Simulation

Status

- Refactored the single-file implementation into a modular package layout
- Separated robot dynamics, navigation algorithms, sensing, obstacles, scenarios, metrics, and simulation runner
- Preserved the legacy public API through `mfinav.__init__` and the compatibility module `mfinav.double_integ`
- Re-ran the benchmark suite and the non-convex diagnostic to confirm behavior was preserved

New package layout

The implementation is now split into:

- `models/` for robot dynamics
- `navigators/` for magnetic-field-inspired and APF logic

- `obstacles/` for obstacle geometry and queries
- `sensing/` for local observation logic
- `scenarios/` for benchmark definitions
- `metrics/` for evaluation
- `sim/` for generic simulation
- `config/` and `utils/` for shared configuration and math helpers

Why this change matters

This refactor is intended to make future work much cleaner:

- the current double-integrator model is now isolated from the navigation logic,
- future models such as mobile robots or drones can be added without duplicating the navigation code,
- future navigation algorithms can be added as separate navigators and compared in the same benchmark framework.

Observed outcome

After the refactor:

- the polygon benchmark results were preserved,
- the non-convex diagnostic results were preserved,
- the repository is now structurally ready for multiple robot models and multiple navigation methods.

Entry 2026-06-19: Stronger Benchmark Environments With True Facing Concavity

Status

- Expanded the benchmark set beyond the earlier easy convex and away-facing concave cases
- Added stronger multi-obstacle and truly concave-facing environments
- Updated the benchmark plotting script to handle a larger scenario set cleanly

New scenarios

The benchmark suite now includes:

- `slalom_multi_convex`: alternating convex obstacles forming a repeated corridor / slalom problem
- `facing_u_shape`: a non-convex U-shaped obstacle whose cavity faces the approaching robot
- `mixed_concave_field`: a true concave-facing obstacle followed by additional downstream obstacles

Why this matters

The earlier non-convex benchmark used a cavity that faced away from the robot, so although the obstacle was mathematically non-convex, the robot effectively interacted with it more like a convex obstacle from the approach direction. The new environments are more faithful tests of the intended non-convex navigation challenge.

Observed outcome

With the stronger environment set:

- both paper-style modes still succeed on the original `single_convex` and `convex_cluster` cases,
- the paper-style geometric controller succeeds on the new `facing_u_shape` case,
- the new `slalom_multi_convex` and `mixed_concave_field` cases are harder and expose remaining limitations,
- the APF and pragmatic baselines remain clearly weaker in the more complex new environments.

Interpretation

This benchmark update is useful because it separates:

- cases where the current implementation is already robust,
- from cases that still require more work in repeated-obstacle and truly concave-facing navigation.

Entry 2026-06-19: Initial 3D Double-Integrator Milestone

Status

- Added an initial 3D path alongside the existing 2D implementation
- Added 3D math helpers, sphere obstacles, 3D benchmark scenarios, and a dedicated 3D benchmark script
- Generalized simulation history and metric computation so 2D and 3D trajectories can share the same runner and evaluator
- Re-ran the existing 2D benchmark suite to confirm the old behavior was preserved

Implemented 3D components

The first 3D milestone includes:

- `SphereObstacle` for simple 3D obstacle geometry
- `MagneticFieldNavigator3D` for 3D magnetic-field-inspired navigation
- `benchmarks3d.py` for a first 3D scenario set
- `run_benchmarks_3d.py` for 3D evaluation and projection plots

Benchmark outcome

The tuned 3D benchmark run now shows:

- `single_sphere`: successful paper-style 3D navigation

- `double_sphere_gap`: successful traversal of a narrow passage between two spheres
- `sphere_cluster`: successful repeated avoidance through a cluttered multi-sphere field
- `offset_cavity`: successful navigation around a reachable non-convex-like sphere composition
- `archway`: successful navigation through a non-convex archway assembled from spheres

The 3D APF comparison remains much weaker:

- it collides in `single_sphere`, `double_sphere_gap`, `sphere_cluster`, and `offset_cavity`
- it reaches `archway`, but with a much longer and less efficient path than the magnetic-field-inspired controller

Added stress case

To keep the benchmark suite honest, a separate stress scenario set was added under `make_stress_scenarios_3d()`.

- `wide_cavity_trap`: a deeper composed-sphere cavity that still traps the current purely local 3D MFI controller

Interpretation

This is a stronger 3D milestone outcome:

- the 3D stack is alive and operational,
- the 2D stack remains preserved,
- the paper-style 3D MFI controller now works on simple, narrow-passage, cluttered, and reachable non-convex-like benchmark cases,
- APF provides a useful baseline comparison but is clearly less robust in these local-sensing 3D scenarios,
- and deeper trap-like cavities remain an open limitation for the current purely local formulation.

Entry 2026-06-19: Stabilized 3D Geometric Benchmark Variant

Status

- Added a dedicated `make_paper_geometric_3d_config()` preset
- Extended the 3D benchmark and interactive HTML viewer to compare `MFI-PD`, `MFI-Geometric`, and `APF`
- Reworked the 3D geometric goal controller so it no longer blows up in cluttered scenes
- Re-ran the 2D benchmark suite to confirm the new 3D geometric work did not change 2D behavior

Main controller change

The first 3D geometric attempt copied the spirit of the 2D turning term too literally and was numerically unstable. The revised 3D geometric logic now uses:

- geometric-style far-field guidance outside the obstacle interaction zone,

- PD-style goal attraction once the sensed obstacle distance is within r_l,
- a 3D lateral turning direction given by the component of the goal direction orthogonal to the current velocity direction,
- an angle-weighted turning command rather than a permanently normalized lateral push.

Current benchmark outcome

With the stabilized 3D geometric variant:

- `single_sphere`: success
- `double_sphere_gap`: success
- `sphere_cluster`: success
- `archway`: success
- `offset_cavity`: still fails by collision

Interpretation

- the 3D geometric benchmark path is now stable enough to keep in the default comparison,
- it is meaningfully stronger than the first unstable 3D geometric draft,
- but it is still not as robust as the 3D paper-style PD variant in the offset cavity case.

Entry 2026-06-19: More Faithful 3D Geometric Steering via Skew-Symmetric Structure

Status

- Removed the temporary "switch to PD inside obstacle range" idea from the 3D geometric path
- Replaced it with a more faithful skew-symmetric steering construction
- Re-ran both the 3D and 2D benchmark suites after the change

Main change

For the far-field 3D geometric branch, the steering command now uses the nested skew-symmetric structure built from the current motion direction l_a and the goal direction l_g:

$$F_{g,\text{turn}}^{3D} = -k_{\text{geom}} \, [l_a]_\times \, ([l_a]_\times l_g) \tag{78}$$

which is equivalent to the lateral projection

$$F_{g,\text{turn}}^{3D} = k_{\text{geom}} \left(I - l_a l_a^\top \right) l_g \tag{79}$$

up to the chosen sign convention inside the controller implementation.

This keeps the geometric steering term inside the same cross-product / skew-symmetric family as the paper's 3D vector formulation, instead of falling back to a PD branch during obstacle interaction.

Current benchmark outcome

With this more faithful 3D geometric benchmark variant:

- `single_sphere`: success
- `double_sphere_gap`: success
- `sphere_cluster`: success
- `archway`: success
- `offset_cavity`: still fails

Interpretation

- this version is more faithful to the paper's 3D control structure than the temporary hybridized 3D geometric experiment,
- it preserves the current 2D benchmark behavior,
- and it gives a usable 3D geometric comparison path even though the offset cavity case remains unsolved.

Entry 2026-06-19: 3D Extruded Polygon Obstacles for Convex and Non-Convex Benchmarks

Status

- Added `PrismObstacle` as an extruded-polygon 3D obstacle model
- Extended the 3D benchmark suite with convex and non-convex prism environments
- Updated the interactive HTML viewer so prism obstacles appear as 3D wireframe solids

Model choice

Instead of introducing a general triangle mesh pipeline, the 3D environment now uses a simpler and more reusable representation:

- a planar polygon footprint in (x, y),
- extruded between $z_{\min}$ and $z_{\max}$.

This preserves the spirit of the earlier 2D polygon benchmarks while keeping the obstacle interface compatible with the local closest-point sensing model.

New benchmark environments

The 3D default benchmark suite now additionally includes:

- `convex_prism_block`: a single convex rectangular prism crossing the direct path
- `nonconvex_prism_u`: a non-convex U-shaped prism whose opening truly faces the robot

Current benchmark outcome

With the current controllers:

- `paper_pd_3d` succeeds on both prism environments
- `paper_geometric_3d` succeeds on both prism environments
- `apf_3d` succeeds on the convex prism block but still fails on the non-convex U-prism case

Interpretation

- the 3D benchmark suite is no longer restricted to sphere-only environments,
- convex and non-convex obstacle structure can now be studied directly in 3D using polygon-derived solids,
- and the prism model is a better bridge toward future robot and environment extensions than sphere-only approximations.

Entry 2026-06-19: Added Legacy Comparison Navigators

What changed

- Added `HaddadinNavigator` in `src/mfinav/navigators/baselines.py`
- Added `SabattiniNavigator` in `src/mfinav/navigators/baselines.py`
- Extended both `scripts/run_benchmarks.py` and `scripts/run_benchmarks_3d.py` to include these baselines

Legacy source anchors

- `nodes/collision_avoidance_new.py`, branches `field == 3` and `field == 4`
- Cross-check against `nodes/collision_avoidance_okt.py`, which contains the same branch structure with slightly different hard-coded gains and cutoffs

Porting notes

- In the ROS repository these methods were implemented as obstacle-only terms added on top of the same go-to-goal node, so the clean Python port keeps the existing goal controller and swaps only the obstacle-response term.
- The old ROS snapshots do not fully agree on exact scalar gains, distance cutoffs, or whether an extra factor of 10 multiplies the Haddadin branch. The clean Python port therefore preserves the branch logic and geometry, while using stable gains tied to the current benchmark framework.
- The old ROS code used an `obs_central` topic. In the clean Python port this is approximated by the centroid of the nearest active obstacle: exact center for circles and spheres, polygon centroid for 2D polygons, and footprint-centroid plus mid-height for prisms.
- The Chang gyroscopic branch from the ROS repository was *not* ported in this change because the legacy implementation outputs only a planar angular velocity command for unicycle-like robots, not a fully actuated double-integrator acceleration.

Implemented formulas

For Haddadin, the obstacle term uses the sensed obstacle vector r_o, a local obstacle-center proxy m_x, goal vector r_g, and robot velocity v:

$$\hat{l}_a = \frac{v}{\|v\|}, \tag{80}$$

$$d = \|r_o\|, \tag{81}$$

$$\hat{g} = \frac{r_g}{\|r_g\|}, \tag{82}$$

$$d_v = m_x - (m_x^\top \hat{g})\hat{g}, \tag{83}$$

$$\hat{t} \propto \left(\frac{-r_o}{d}\right) \times \frac{d_v \times r_g}{\|d_v \times r_g\|}, \tag{84}$$

$$B = \frac{\hat{t} \times v}{\|v\| d^2}, \tag{85}$$

$$F_{\text{Haddadin}} = k_H \left(\hat{l}_a \times B\right) \|v\|. \tag{86}$$

For Sabattini, the obstacle term uses the goal vector r_g, velocity v, obstacle vector r_o, and a damping term based on the signed approach speed:

$$u_t = 0.1\, r_g, \tag{87}$$

$$w = u_t - \text{proj}_v(u_t), \tag{88}$$

$$u_g = -k_S \frac{w}{\|w\|}, \tag{89}$$

$$\sigma = \begin{cases} 1, & v^\top r_o \geq 0, \\ 0, & v^\top r_o < 0, \end{cases} \tag{90}$$

$$F_{\text{damp}} = -0.05\, s\, \frac{E_0}{\|r_o\|} \left(|v^\top r_o| + e^{-|v^\top r_o|}\right) \frac{r_o}{\|r_o\|}, \tag{91}$$

$$F_{\text{Sabattini}} = \sigma\, (u_g + F_{\text{damp}}), \tag{92}$$

where $E_0 = 0.1\, d_{g,0}^2$ uses the initial goal distance and $s \in \{+1, -1\}$ follows the sign convention from the ROS code.

Entry 2026-06-19: Added Differential-Drive Kinematic Model

What changed

- Added `DifferentialDriveState` and `DifferentialDriveModel` in `src/mfinav/models/differential_drive`
- Added `simulate_differential_drive` in `src/mfinav/sim/differential_drive.py`
- Added 2D nonholonomic benchmark script `scripts/run_benchmarks_diff_drive.py`

Motivation

- The repository previously benchmarked only fully actuated double-integrator motion.
- A differential-drive model is needed to test the same navigation algorithms on a nonholonomic planar robot while keeping the sensing and obstacle abstractions unchanged.

Modeling choice

The navigation algorithms in this repository output a planar guidance vector $u \in \mathbb{R}^2$. For the differential-drive robot, this vector is interpreted as a desired heading and speed command rather than as a physical acceleration.

Let the robot state be

$$x = \begin{bmatrix} p_x & p_y & \theta & v & \omega \end{bmatrix}^\top$$

with planar position p, heading θ, linear speed v, and angular speed ω. Given guidance vector u, define

$$\theta_d = \operatorname{atan2}(u_y, u_x), \tag{93}$$
$$e_\theta = \operatorname{wrap}(\theta_d - \theta), \tag{94}$$
$$\omega_c = \operatorname{sat}_{\omega_{\max}}(k_\theta e_\theta), \tag{95}$$
$$v_c = \operatorname{sat}_{v_{\max}}(k_v \|u\|)\ \max(0, \cos e_\theta). \tag{96}$$

The kinematic propagation is then

$$\dot{p}_x = v_c \cos\theta, \tag{97}$$
$$\dot{p}_y = v_c \sin\theta, \tag{98}$$
$$\dot{\theta} = \omega_c. \tag{99}$$

Interpretation

- This is a tracking wrapper around the existing navigation field, not a re-derivation of the original papers for differential-drive actuation.
- The same local sensing and obstacle models are reused, so differences in benchmark behavior mainly reflect the effect of the nonholonomic kinematics and heading-tracking layer.
- Differential-drive benchmarks are therefore useful for qualitative portability studies and algorithm comparison, but should not be interpreted as a paper-faithful nonholonomic controller unless a dedicated derivation is added later.

Entry 2026-06-19: Added 3D Quadrotor Dynamics Model and Benchmark Path

What changed

- Added `QuadrotorState` and `QuadrotorModel` in `src/mfinav/models/quadrotor.py`
- Added `simulate_quadrotor` in `src/mfinav/sim/quadrotor.py`
- Exported the quadrotor model and simulator through the package compatibility layer
- Added dedicated 3D quadrotor benchmark script `scripts/run_benchmarks_quadrotor_3d.py`

Motivation

- The clean Python repository already supported 3D point-mass navigation, but not the quadrotor dynamic model used in the ICRA 2019 ROS implementation.
- A dedicated quadrotor model is needed so the same 3D navigators can be tested under a more realistic underactuated vehicle assumption.

Modeling choice

The quadrotor state is

$$x = \begin{bmatrix} p^\top & v^\top & R & \omega^\top & T \end{bmatrix},$$

where $p \in \mathbb{R}^3$ is position, $v \in \mathbb{R}^3$ is linear velocity, $R \in SO(3)$ is attitude, $\omega \in \mathbb{R}^3$ is angular velocity, and T is total thrust.

The high-level navigation layer still outputs a 3D guidance vector $u \in \mathbb{R}^3$. In the quadrotor model this is interpreted as the desired translational navigation force relative to hover:

$$F_Q = u + mge_3.$$

The rigid-body propagation follows the same structure as the legacy ROS quadrotor node:

$$\dot{p} = v, \tag{100}$$

$$\dot{v} = \frac{1}{m}\left(Re_3T + G\right), \tag{101}$$

$$\dot{R} = R\hat{\omega}, \tag{102}$$

$$\dot{\omega} = J^{-1}\left(J\hat{\omega}\,\omega + \tau\right). \tag{103}$$

Low-level actuation assumption

To match the practical quadrotor controller path in the old ROS implementation, the clean Python model uses a thrust plus attitude-tracking layer rather than driving the navigation field directly into rotor speeds.

The total thrust tracks the desired force magnitude:

$$T_d = \|F_Q\|.$$

Desired roll and pitch are computed from the horizontal components of F_Q and the available thrust, with yaw regulated toward zero:

$$\phi_d = -k_f \frac{F_{Q,y}}{\max(T_d, \varepsilon)}, \tag{104}$$

$$\theta_d = k_f \frac{F_{Q,x}}{\max(T_d, \varepsilon)}, \tag{105}$$

$$\psi_d = 0. \tag{106}$$

The torque command is then a PD law on roll, pitch, and yaw:

$$\tau_\phi = -k_\phi(\phi - \phi_d) - d_\phi\omega_x, \tag{107}$$

$$\tau_\theta = -k_\theta(\theta - \theta_d) - d_\theta\omega_y, \tag{108}$$

$$\tau_\psi = -k_\psi(\psi - \psi_d) - d_\psi\omega_z. \tag{109}$$

Rotor speeds are computed through the Pelican-style allocation matrix used in the old ROS code, but the internal benchmark dynamics are propagated with the commanded thrust and torques, matching the legacy non-Gazebo simulation path.

Current benchmark outcome

The first quadrotor benchmark pass is now operational and writes:

- `artifacts/benchmark_comparison_quadrotor_3d.png`
- `artifacts/benchmark_metrics_quadrotor_3d.csv`

At the current tuning level:

- the quadrotor dynamics are stable enough to simulate all current 3D navigators across the default 3D benchmark set,
- the existing 3D double-integrator benchmark path still runs unchanged after the addition,
- the existing 2D benchmark path still runs unchanged after the addition,
- but the quadrotor benchmark results are not yet as strong as the point-mass 3D results and still need dedicated gain tuning, especially in the clustered and cavity-like environments.

Interpretation

- This addition creates the needed structural bridge from the clean 3D navigation layer to the older quadrotor paper and ROS implementation.
- The current quadrotor path is already useful for qualitative comparison and model-portability testing.
- Full paper-level reproduction of the quadrotor results will likely require a second tuning pass that is specific to the quadrotor actuation and benchmark geometry, rather than reusing the point-mass gains directly.

Entry 2026-06-23: Dynamic 2D Obstacle Benchmarks

Summary

- Added a dynamic-obstacle layer for the 2D benchmark path using time-varying translational motion for circles and polygons.
- Extended the simulation runner so obstacle geometry can be updated at each simulation step through a common `set_time()` interface.
- Added a dedicated dynamic benchmark suite with moving circular, convex polygonal, and non-convex polygonal obstacles.
- Added an interactive HTML benchmark report that shows the robot trajectories together with the moving obstacle geometry.

Model extension

For the first dynamic benchmark pass, each obstacle follows a prescribed translational motion:

$$p_O(t) = p_O(0) + v_O t.$$

For circles this updates the obstacle center directly. For polygons it shifts every vertex by the same translation vector:

$$v_i(t) = v_i(0) + v_O t.$$

The navigation algorithms themselves are unchanged. They are evaluated against the obstacle geometry at the current simulation time $t_k = k\,\Delta t$, so the reactive comparison now includes time-varying obstacle motion without introducing any prediction or future-state estimation into the navigators.

Benchmark set

The first dynamic 2D suite contains:

- `moving_circle_crossing`: a circular obstacle sweeping upward across the path,
- `moving_convex_sweeper`: moving convex polygonal obstacles crossing and drifting along the route,
- `moving_u_shape`: a non-convex U-shaped obstacle whose mouth faces the robot while the whole obstacle drifts during the encounter.

Artifacts

The dynamic benchmark writes:

- `artifacts/benchmark_comparison_dynamic_2d.png`
- `artifacts/benchmark_comparison_dynamic_2d.html`
- `artifacts/benchmark_metrics_dynamic_2d.csv`

First result snapshot

In the first dynamic benchmark pass:

- both paper-style MFI variants succeeded in all three moving-obstacle scenarios,
- APF also succeeded in all three current dynamic scenarios,
- Haddadin failed the moving-circle crossing case but succeeded in the moving convex and moving non-convex polygon cases,
- Sabattini failed all three current dynamic scenarios under the present tuning and success criterion,
- and the original static 2D benchmark suite still runs after the dynamic-obstacle extension.

Interpretation

- This is the first benchmark step where the clean repo explicitly evaluates the navigators against moving obstacles rather than only static geometry.
- The current dynamic layer is still kinematic obstacle motion, not deformable or interaction-coupled motion.
- It is already sufficient to compare reactivity and robustness under time-varying obstacle geometry, especially between the local MFI variants and the less reactive baselines.